

Weekly Progress Log

Project Title

Bibliography Manager

Date

2026-09-14

Week

Starting from this week

This document is AI-generated and is intended to provide a concise weekly summary of project progress, key issues, and next steps for review by the developer and the supervising professor.

The project is being developed by a single developer, with direct feedback and review from the supervising professor. No additional team members are involved in the core development workflow at this stage.

1. Project Overview

The Bibliography Manager is a BibTeX management application designed to help researchers and students organize, deduplicate, normalize, and export bibliography entries from multiple files and folders. The system addresses common issues such as duplicate references, inconsistent citation keys, formatting inconsistencies, and large bibliographic collections spread across different directories. At this stage, the application has developed beyond a basic prototype. It includes a core processing engine, a modern graphical user interface, and supporting documentation, and it is close to being ready for academic review by the supervising professor.

2. Current Progress

The following major components have already been implemented and are functioning:

- Core processing engine in `bib.py`
 - Collects BibTeX entries from folders and files
 - Detects duplicates using author, title, year, and DOI-based logic
 - Merges duplicate entries intelligently
 - Regenerates citation keys according to selected naming rules
 - Creates a consolidated master bibliography file
 - Pushes cleaned results back to original folders
 - Generates an HTML report summarizing processing results
- Modern GUI in `bib_gui_modern.py`
 - Sidebar-based control panel
 - Folder and archive browsing
 - Duplicate similarity threshold adjustment

- Citation key policy selection
 - Light and dark appearance modes
 - Pipeline progress tracking and status updates
 - Library browser for reference browsing and metadata handling
 - Duplicate review and fix visualization
- Supporting project files
 - README documentation and setup instructions
 - Sample bibliographic data in `test_data/`
 - Backup generation before modifying original files
 - Generated HTML reports for review and presentation

3. Key Technical Achievements

The project has achieved the following notable milestones:

- Built a working BibTeX collection and normalization pipeline
- Developed duplicate detection logic to reduce redundant references
- Created a modular structure that separates core logic from GUI interaction
- Implemented citation key normalization with multiple naming options
- Added a professional HTML reporting feature
- Improved the interface design and usability compared with earlier versions

4. Current Issue Identified by the Professor

One of the main concerns raised by the professor is citation key readability and collision risk. The current system supports several citation naming schemes, but some short formats based on abbreviated author names and year can become ambiguous when multiple papers from the same author and year exist.

For example:

- `aa-2012`
- `ab-2012`

These entries may look visually similar and can be difficult for a human reviewer to distinguish. Although the system attempts to ensure uniqueness by appending suffixes when necessary, the underlying readability problem remains if the base key is too short or too generic.

This is an important issue because a bibliography manager should not generate keys that are technically valid but visually confusing to the user, particularly in academic work where citation keys are reviewed by supervisors and researchers.

5. Root Cause and Interpretation

The issue originates from the citation key generation logic. In some cases, the code creates very compact author fragments followed by a year and, in some cases, a short title-related token. While this improves brevity, it reduces human recognizability.

The system can avoid exact duplicates by adding suffixes such as **a**, **b**, or **c**, but the more important issue is whether the generated citation key remains meaningful enough for a person to understand and identify the paper quickly.

This means the professor's concern is valid and should be treated as a design requirement rather than a minor issue.

6. Recommended Fix

To address the professor's concern, the system should adopt a clearer and more robust citation key naming policy. The most suitable approach is to maintain a human-readable structure while preserving uniqueness.

A recommended final key format is:

- `{author}-{year}-{titleword}`

This format is readable, relatively short, and reduces confusion between papers by introducing a meaningful keyword from the title.

An alternative compact version may be:

- `{authorstem}{year}-{titleword}`

However, pure short formats such as only author and year, or highly abbreviated author strings, should be avoided as the default because they are too easy to confuse.

7. Current Status and Decision

The project is no longer in an early development stage. It is close to being a complete working prototype and has already demonstrated the core functional requirements:

- Bibliography collection
- Duplicate detection and merging
- Key normalization
- Output generation
- Automated backup and reporting

The next critical step is not to add entirely new features, but to finalize the citation key strategy and ensure that it is applied consistently across the code, interface, and documentation.

8. Proposed Next Step: Web Deployment for Remote Testing

To improve testing efficiency and reduce dependency issues during project review, it is proposed that the application be adapted from a desktop-only workflow into a browser-based deployment model.

This approach would allow the professor to access the application through a web browser without needing to install a Mac package or run a local desktop build. Since the project is managed by a single developer and is reviewed primarily by the supervising professor, a web deployment would provide a simpler and more practical testing workflow.

The project could be hosted on a local personal computer, a Raspberry Pi at home, or a small VPS. The professor would only need to open the hosted web application and test it remotely. This would make it easier

to receive feedback quickly and update the project without distributing a new Mac build for every iteration.

This proposal is practical and efficient because the project already contains a Streamlit web app in `bib_streamlit.py`, which can serve as the foundation for browser-based testing. A Docker container can further standardize the deployment environment and improve consistency across different systems.

A practical hosting plan for this project is as follows:

- Host the app locally on the developer's Raspberry Pi or personal computer at home
- Or use a small VPS for a more stable and accessible remote deployment if required
- Use the web interface for professor testing and issue reporting

For this project, the Raspberry Pi is a strong near-term option because it is low-cost, available locally, and suitable for lightweight remote testing. A small VPS remains the more stable long-term option if the application needs to be accessed more reliably from outside the home network.

9. Deployment Architecture

The proposed architecture for future development is as follows:

- Frontend: Streamlit web interface
- Backend: Python processing engine and bibliography logic
- Data handling: uploaded `.bib` files or local folder input
- Outputs: processed bibliography file, HTML report, duplicate summaries, and normalized citation keys
- Deployment: Docker container or direct Streamlit service
- Access model: the professor uses a web browser only

A Docker-based deployment would help maintain consistency across development and testing environments. The container can be configured to run the Streamlit app on a standard port such as 8501 and can be hosted on a local machine, a Raspberry Pi, or a small VPS.

This would allow frequent updates to be deployed quickly and tested remotely without packaging a Mac-specific application for every iteration.

10. Cost and Feasibility of Hosting

For this final-year project, the use of a small VPS or a local Raspberry Pi hosting setup is feasible and practical. A small VPS is usually inexpensive compared with the time spent packaging and redistributing desktop builds for different operating systems, while a Raspberry Pi is even more economical if the goal is simply to host the web app locally.

From a project management perspective, the cost of a small VPS can be justified as a development and testing infrastructure expense because it directly supports:

- remote access for the supervising professor
- easier testing without local installation
- reduced need to send multiple OS-specific builds
- faster iteration and feedback during project improvement

Whether the cost is claimable depends on the project policy and institutional guidelines. If the hosting cost is used strictly for project testing, demonstration, and academic review, it is reasonable to describe it as a

project-supporting infrastructure expense. Final claim eligibility should be confirmed with the professor or project coordinator before purchase.

If the institution permits project-related software hosting costs, a low-cost VPS is a reasonable and justifiable expense. If approval is required, the project should document the need for a hosted testing environment before proceeding.

11. Final Recommendation

The next step should be to shift from desktop-only delivery toward browser-based deployment, starting with a Streamlit web app and optionally a Docker container. This would enable more efficient testing and feedback from the professor, reduce installation issues, and provide a more professional demonstration workflow.

This is especially appropriate because the project is now functionally mature enough that the biggest remaining need is better accessibility, rapid iteration, and remote validation rather than further basic feature creation. In this final-year project context, the developer is a single person, and the key stakeholders are the developer and the supervising professor; therefore, a simple web-hosted testing flow is the most practical and efficient next step.

12. Conclusion

The project has reached a strong functional stage, and the main remaining objective is to improve deployment and feedback workflows. A web app or Dockerized deployment would significantly improve the demonstration and review process, especially when the professor is using a Mac and the developer is working on a Windows PC.

Given the project size and limited stakeholder group, it is reasonable to propose a local Raspberry Pi or a small VPS as the next hosting option for remote testing and demonstration. This should be treated as part of the project roadmap for the upcoming weeks.

This weekly logbook entry marks the beginning of a structured weekly documentation process and will be stored in the Weekly Logbook folder for ongoing progress tracking.

Note: This document was generated with AI assistance to summarize project progress, key issues, and the proposed development strategy.